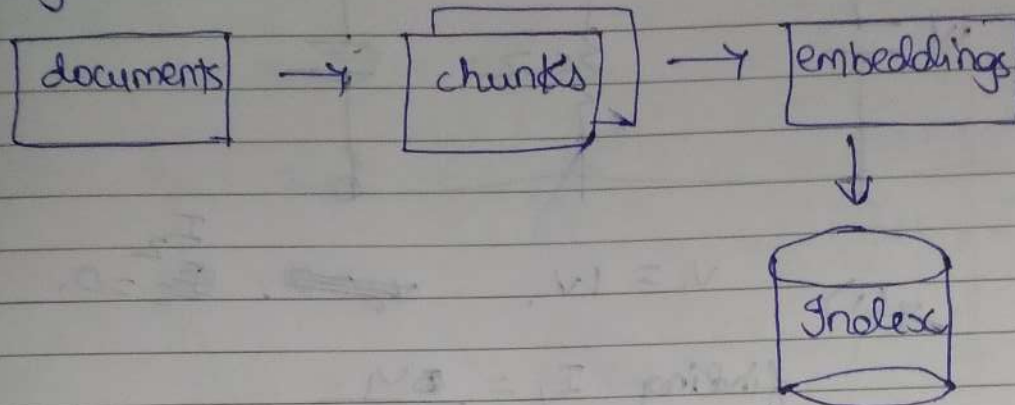


Our first RAG Pipeline.

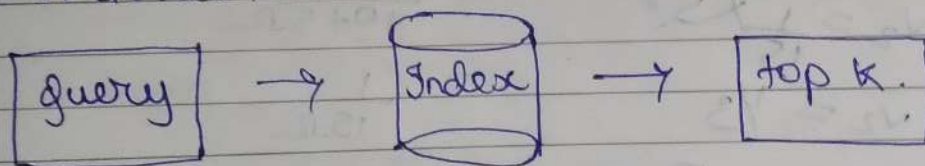
RAG works on the concept of cosine similarity.

```
def cosine-similarity(query, document):  
    # Tokenize and convert to lower case.  
    query_tokens = query.lower().split(" ")  
    document_tokens = document.lower().split(" ")  
  
    # Create counter for query and document  
    we will use counter as embeddings.  
    query_counter = Counter(query_tokens)  
    document_counter = Counter(document_tokens)  
  
    # Calculate dot product  
    dot_product = sum(query_counter[token] *  
        document_counter[token] for token in  
        query_counter.keys() & document_counter.keys())  
  
    # Calculate magnitude  
    query_magnitude = math.sqrt(sum(query_counter  
        [token]**2 for token in query_counter))  
    document_magnitude = math.sqrt(sum(document_counter[token]  
        **2 for token in document_counter))  
  
    # Calculate similarity  
    similarity = dot_product / (query_magnitude * document  
        magnitude)  
  
    return similarity
```

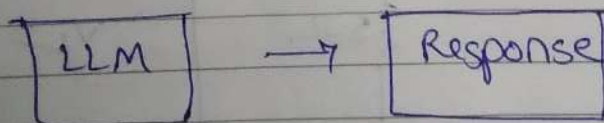
Ingestion :-



Retrieval :-



Generation :-



Auto - merging Retriever

Auto - merging retrieval aims to combine information from multiple sources or segments of text to create a more comprehensive and contextually relevant response to a query.

a) Let's understand the augmentation and generation.

This process involves integrating the insights gleaned from various resources, ensuring accuracy and relevance, and crafting a response that is not only informative but also aligns with the user's original query, maintaining a natural and conversational tone.

B) Let's understand the retrieval process

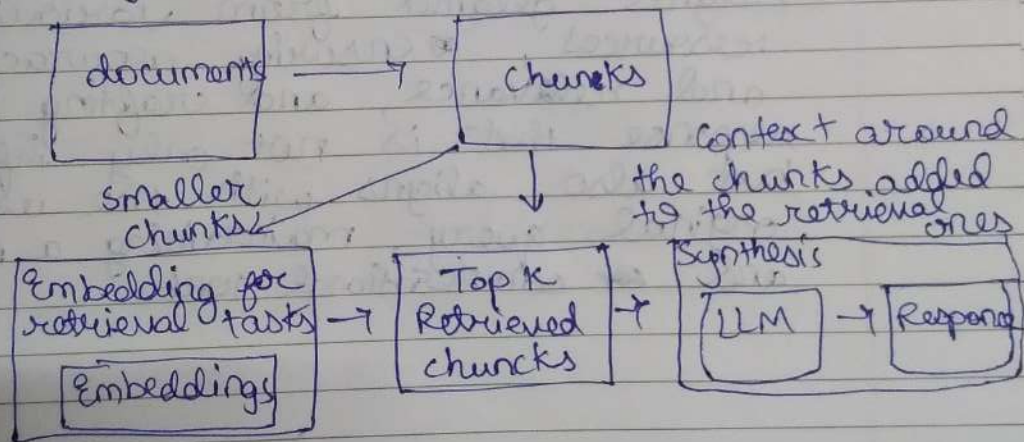
Advantages:-

- Simplicity and efficiency
- uniformly in data handling

disadvantages:-

- Limited contextual understanding
- potential for suboptimal responses.

Sentence - window Retrieval / small to large chunking



during retrieval, we retrieve the sentences that are most relevant to the query via similarity search and replace the sentence with the full surroundings context.

3. Memory and computational resources.

4. Task requirements.

5. Experimentation.

6. Overlap consideration.

3. Embedding

After chunking we convert it into vector embeddings. Vector embeddings are numerical representations of the data.

1. Frequency based embedding

BOW

① TF-IDF

② N-GRAMS

2. Neural Network based embeddings.

① word2Vec

② fastText

③ BERT

④ ELMO

⑤ OpenAI embedding

⑥ Gemini embedding

4. Vector Embedding Indexing.

A vector database indexes and stores vector embeddings for fast retrieval and similarity search.

B.) Let's

Adv

• Sim

• Uni

dis

• Limi

• pote

Sente

lat

Embe

retrie

Em

durin

that

qu

re

f

A.) Ingestion process

1. document

In typical RAG pipeline, we have Knowledge ~~resources~~ sources such as local files, CSV, JSON, PDFs, Docs, Web pages, XML, databases, etc.

2. chunking:

we collect the data from various sources, split the data, into the chunks.

• why chunking is required?

we can feed the entire document also but why we are just passing so here the reason is.

① LLM will not be overloaded with information and

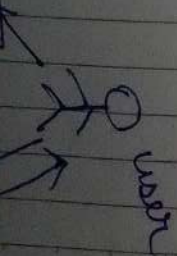
② Even it is having some limits in terms of tokens.

Based on these factor you can decide the size of chunk.

① data characteristics

② Retriever constraints.

Architecture



A.) Ingestion

1. document

in type
knowledge
files, CSV,
XML,

2. chunking

we collect
sources

why chunk

we can
but will
so have

① LLM
inform

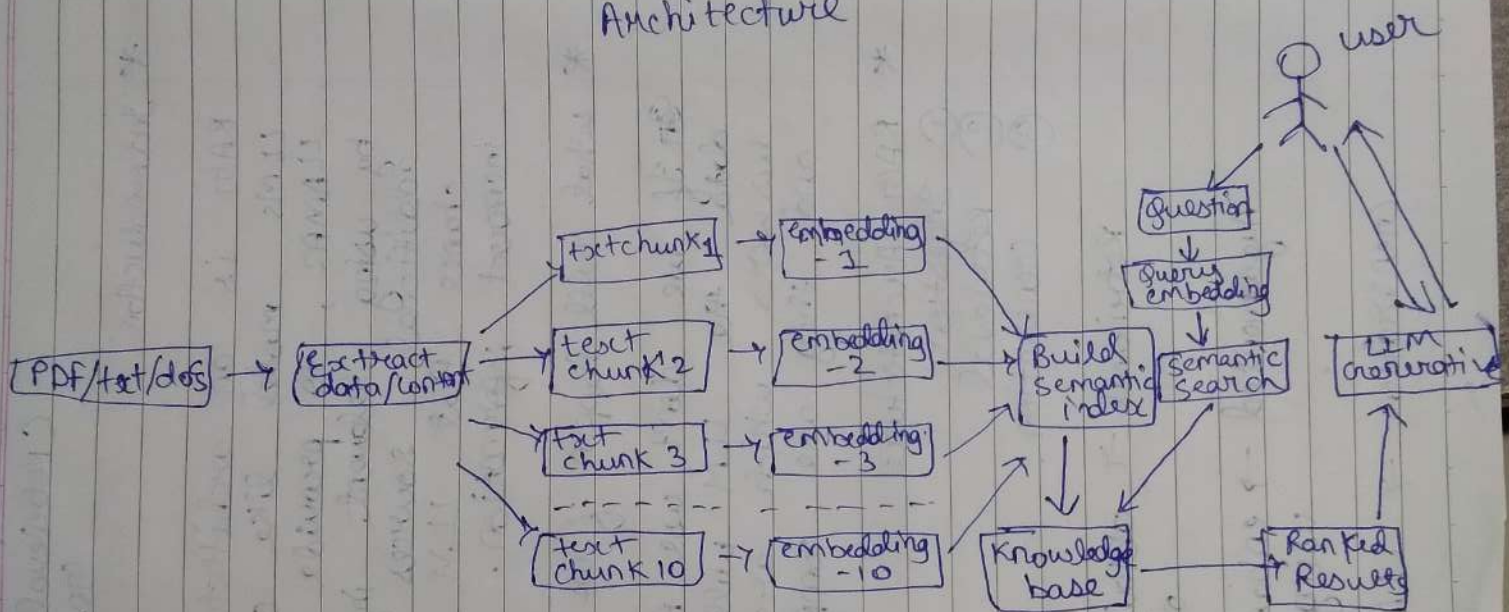
② even
term

Based
the

① data

② retrieve

Architecture



- ① Ingestion
- ② Retrieval
- ③ Generation.

RAG (Retrieval Augmented Generation)

* Introduction

RAG is an architecture used to help LLMs model like GPT-4, Gemini, Gemma, LLaMA2, to provide a better response by using relevant information from additional sources and reduce the chance that LLM will lead to incorrect information.

* What will happen if we not use RAG pipeline?

~~So~~ If we don't use RAG pipeline for our data the LLM will give hallucinating answers which are not accurate and they are completely bluff.

* RAG Architecture

- ① Ingestion.
- ② Retrieval
- ③ Generation.

Retrieval → obtaining data from database

Augmented → enhancing information using LLM.

Generation → LLMs :- e.g. Gemma,